

NetConfigGuard: Model Assisted Verification and Repair of LLM-Generated Network Security Configuration

Name: Hasnat Jamil Bhuiyan

Introduction:

Large Language Models (LLMs) are increasingly used to assist technical tasks such as code generation, system administration and security configuration. A user can now ask an LLM to generate SSH settings, firewall rules, Nginx reverse proxy configurations, Docker Compose files or Kubernetes Network Policy files in a matter of seconds. This convenience is useful but it also creates a serious network security risk: a configuration can look correct and still expose services, weaken authentication or permit excessive network access. This problem is important because network and security configurations are often directly connected to the attack surface of a deployed system. For example, an SSH configuration that allows root login, a firewall rule that opens sensitive ports to 0.0.0.0/0, a Docker Compose file that exposes a database or an Nginx configuration missing security headers may all be functional while remaining insecure. Since LLMs are trained on large public corpora that may contain outdated or convenience oriented examples, their generated configurations should not be trusted without verification. This project proposes NetConfigGuard, a lightweight model assisted framework for checking LLM generated network configurations before deployment. The general idea is to combine a model-based security analyzer with a deterministic rule-based verification layer. The model component identifies possible security issues and explains why they matter while the rule component checks known high risk patterns and produces evidence based findings. The final output is a report containing severity labels, affected configuration lines or fields and repair suggestions. The project focuses on the research question: How can we detect and reduce insecure settings in LLM-generated network configurations using a lightweight and explainable approach? NetConfigGuard focuses on five configuration categories: SSH, firewall rules, Nginx, Docker compose and Kubernetes NetworkPolicy. These categories represent authentication, service exposure, web security, container networking and cluster level access control.

Related Work:

Recent papers from major security conferences motivate this project. PromSec studies the problem that LLM-generated source code may be functional but still contain vulnerabilities and proposes prompt optimization to improve both security and functionality [1]. This is closely related because NetConfigGuard also treats LLM output as a technical artifact that requires security checking. The main difference is that PromSec focuses on Python and Java source code while this project focuses on network and security configuration files such as SSH, firewall, Nginx, Docker Compose and Kubernetes policies. NetConfigGuard also uses a simpler model-plus rule verification design instead of a graph-based code repair framework.

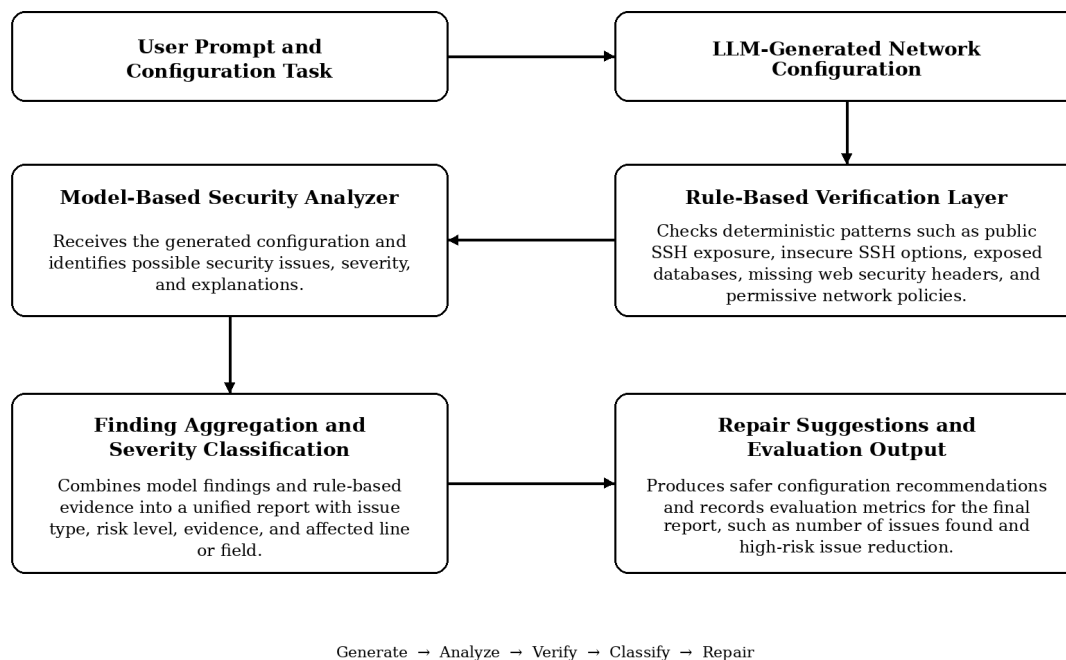
Large Language Model guided Protocol Fuzzing, also known as CHATAFL, uses LLMs to support protocol fuzzing by extracting protocol grammars, enriching seed inputs and guiding exploration of protocol states [2]. This paper is relevant because it shows that LLMs can help reason about structured network related artifacts. However, CHATAFL dynamically tests protocol implementations while NetConfigGuard statically checks generated configuration files before deployment. In other words, CHATAFL applies LLMs to network protocol fuzzing, while this project applies LLMs and rules to network configuration verification.

PentestGPT evaluates and harnesses LLMs for automated penetration testing [3]. It shows that LLMs can help with security sub tasks such as interpreting results and suggesting next steps but they also struggle with maintaining context across complex workflows. This supports the motivation for NetConfigGuard: LLMs can assist with cybersecurity but their output needs structure, verification and guardrails. Unlike Pentest, which focuses on interactive penetration testing, this project targets the narrower and more feasible problem of detecting insecure settings in generated network configurations. Together, these works show that LLMs are useful but imperfect tools for cybersecurity . NetConfigGuard builds on this direction while targeting a distinct gap; automatic verification and repair guidance for LLM generated network security configurations.

Approach:

The proposed system, NetConfigGuard, is designed as a hybrid verification framework for LLM-generated network configurations. The system follows a pipeline with five main stages: configuration input, model based analysis, rule based verification, finding aggregation and repair suggestions. Figure-1 illustrates the overall architecture:

Figure 1. Architecture of NetConfigGuard



The first stage is configuration input. The project uses a dataset of LLM-style network configuration files covering five categories commonly used in network and cloud environments: SSH server configurations, firewall configurations, Nginx reverse proxy or web server configurations. Docker Compose files, Kubernetes Network Policy or related access control configurations. These categories were chosen because they are security relevant and feasible to analyze in a lightweight research prototype. The second stage is the model based security analyzer. This component receives a generated configuration file and attempts to identify possible security issues. The model based analyzer semantically provides

meaningful findings such as issue descriptions, severity labels, risks and candidate fixes. In the final implementation, the analyzer supports prompt mode, manual mode and Gemini API mode. The API based experiment used Gemini 2.5 flash, which was the available Gemini model in the free tier environment used in this project. This directly addresses the need to specify the LLM model used in the experiments.

The third stage is the rule based verification layer. This component checks for explicit high risk patterns that can be identified deterministically. Rule-based verification is important because it provides reliable and explainable evidence even when the model based component is uncertain. The implemented rules check insecure SSH authentication settings, broad firewall exposure, missing Nginx hardening headers, unsafe Docker Compose networking or privilege settings and insecure Kubernetes networking or security Context settings.

The fourth stage is finding aggregation and severity classification. This stage combines the outputs of the model based analyzer and the rule based verifier into a unified security report. Findings are de-duplicated using fields such as file name, configuration type and issue identifier. If both the model analyzer and rule checker detect the same issue, the final detector label becomes hybrid. This avoids double counting while still showing when both components agree.

The fifth stage is a repair suggestion. After a finding is identified, the system produces a safer alternative or a mitigation recommendation. The repair mechanism in this project is intentionally lightweight. It does not attempt full automatic correction of every configuration. Instead, it provides practical suggestions such as disabling root login, restricting public network exposure, enabling HTTPS redirection, removing exposed database ports or limiting Kubernetes ingress rules. This keeps the system explainable and reduces the risk of introducing new configuration errors during automated repair.

The main contribution of the project is therefore the verification workflow which is detecting insecure LLM generated network configurations and providing interpretable guidance before deployment. The hybrid design balances practicality with research value. A purely rule based scanner would be easier to evaluate but would not fully reflect the AI-security setting of the project. A purely model-based analyzer would be more flexible but harder to verify reliably. By combining the two, NetConfigGuard maintains a clear AI component while still producing evidence-based security checks.

Implementation:

NetConfigGuard was implemented as a Python research prototype that operationalizes the five stage design in Figure-1. The implementation uses a shared finding schema for all detectors so that rule-based findings, model-generated findings and merged hybrid findings can be compared consistently. Each finding records the affected file, configuration type, issue identifier, severity, evidence, risk explanation, suggested fix, detector label and source. The rule verifier implements deterministic checks for the five target configuration categories. The model analyzer implements prompt based, manual and Gemini API based analysis modes. Prompt mode creates structured prompts that can be submitted to an LLM, manual mode reads structured model findings from JSON and Gemini mode sends configuration text to Gemini 2.5 flash through the Gemini API. API keys are not stored in the code; the Gemini mode reads the key from an environment variable and falls back safely if the key or quota is unavailable. The hybrid detector merges model and rule findings, removes duplicates and labels overlapping findings as hybrid. The

evaluation component then produces CSV outputs, summary metrics and bar charts for the final report. Automated tests were included for representative rule checks and the final test run completed successfully with 6 passing tests. This implementation is intentionally lightweight. It demonstrates an explainable research prototype for verifying LLM-generated network security configurations.

Experiments and Results:

Dataset:

The experiment used 30 sample network configuration files. The dataset was organised across five configuration categories with each category containing three baseline or insecure examples and three secure or security aware examples. This design allows comparison between risky configurations and safer configurations while keeping the experiments feasible.

Table1: Dataset Composition

Configuration Type	Baseline/Insecure Files	Secure/Security Aware Files	Total Files
SSH	3	3	6
Firewall	3	3	6
Nginx	3	3	6
Docker Compose	3	3	6
Kubernetes	3	3	6
Total	15	15	30

Each configuration file was processed through the NetConfigGuard pipeline. The rule checker scanned all files, the model analyzer used Gemini 2.5 Flash through the Gemini API where free-tier quota allowed successful calls, the hybrid detector merged overlapping results and the evaluation module generated summary tables and charts.

Model and Evaluation Setting:

The specific LLM model used for the API based model assisted experiment was Gemini 2.5 Flash. The tool was also designed to support configurable model names but the reported API based results in the report use Gemini 2.5 Flash. The rule based verifier was applied to all 20 configuration files. The Gemini free tier limited the number of successful API calls during the full run, so the model assisted results should be interpreted as preliminary API based findings rather than exhaustive model coverage over every file. Successful Gemini calls produced 270 findings which were merged with deterministic rule findings by the hybrid detector.

Overall Findings:

The final run produced 60 rule based findings and 27 model based findings. After de-duplication, the hybrid detector produced 81 unique findings. Of these, 54 were rule only findings, 21 were model only

findings and 6 were findings detected by both the rule checker and Gemini model analyzer.

Table 2: Detector output Summary

Detector Category	Number of Findings
Rule only findings	54
Model only findings	21
Hybrid findings detected by both	6
Total unique hybrid findings	81

The result shows that the deterministic rule checker provided broad coverage for known high risk patterns, while Gemini 2.5 Flash contributed additional model based findings and confirmed some rule based detections. The hybrid stage prevented duplicate counting and produced a clean final result set.

Issues by Configuration Type:

Figure 2 shows the number of findings by configuration type. Docker Compose produced the largest number of issues followed by firewall rules, SSH, Nginx and Kubernetes.

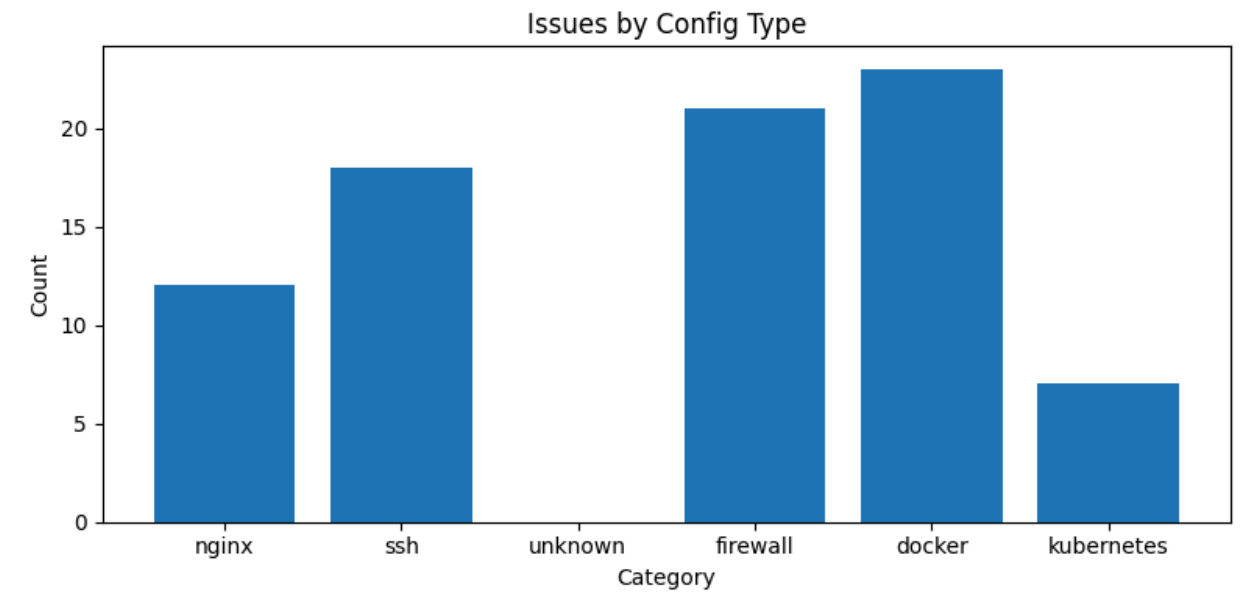


Figure -2 : Issues by configuration type

Configuration Type	Findings
Docker Compose	23

Firewall	21
SSH	18
Nginx	12
Kubernetes	7
Total	81

Docker Compose had the highest number of findings because the implemented checks and Gemini findings identified common container security problems, including privileged containers, host networking, public database or administrative ports , weak secrets, and missing hardening settings. Firewall configurations also produced many findings because broad allow rules and public exposure of sensitive services are high risk network security patterns. SSH and Nginx produced fewer findings than Docker and firewall rules but still contained important authentication and web hardening issues. Kubernetes produced the smallest number of findings in this dataset.

Issues by Severity:

Figure 3 shows the distribution of findings by severity. High severity findings were the most common, followed by critical and medium findings.

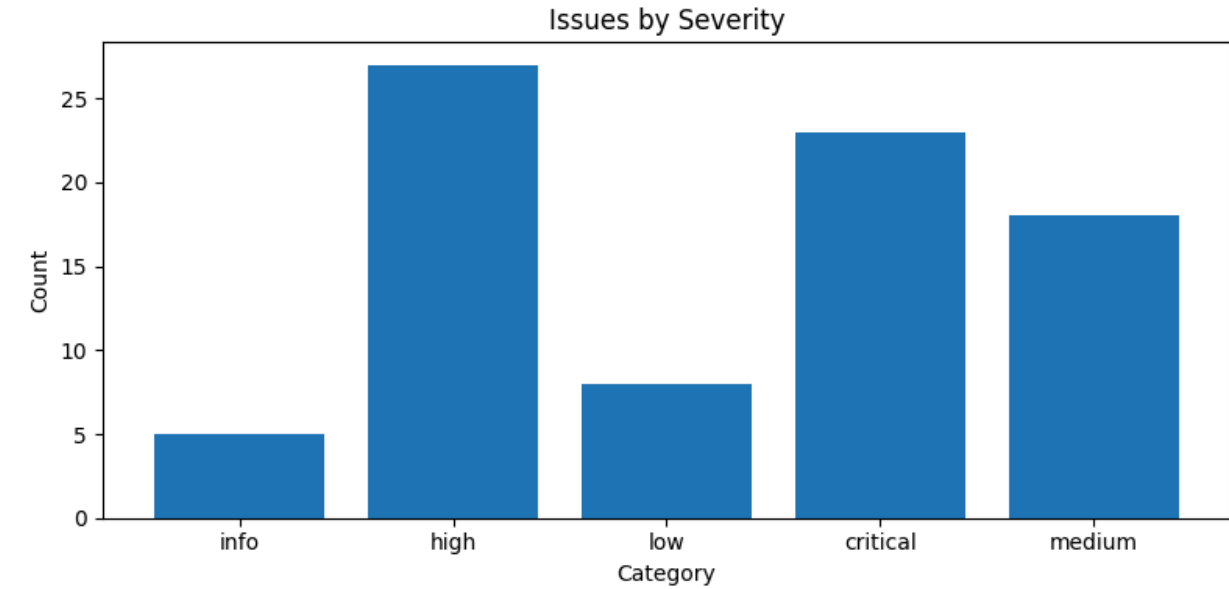


Figure 3: Issues by severity

Table 4: Findings by severity

Severity	Findings
----------	----------

Critical	23
High	27
Medium	18
Low	8
Info	5
Total	81

The results show that many detected issues were security relevant rather than purely informational. The high severity and critical findings include public exposure of services, Docker privileged containers, host networking, public database ports, weak secrets, broad firewall access and insecure SSH options. Medium and low findings included missing hardening headers, HTTP to HTTPS redirection issues, X11 forwarding and related configuration weaknesses. Informational findings were primarily hardening suggestions rather than immediate vulnerabilities.

Baseline vs Secure Configuration Comparison:

The experiment also compared findings across baselines/ insecure files and secure/security aware files. The baselines files generated most of the findings.

Table-5: Baseline vs Secure Configuration findings

File Type	Number of Files	Findings	Average Findings per File
Baseline/Insecure	15	76	5.07
Secure/Security Files	15	5	0.33
Total	30	81	2.70

This result supports the main motivation of the project. Configurations written with insecure or baseline assumptions produced many more findings, while secure/security aware examples produced far fewer. However, the security examples were not necessarily perfect. A small number of findings still appeared, mostly because the tool also reports hardening recommendations. This shows why an additional verification step is useful even when a configuration appears security conscious.

Repair Suggestion Coverage:

Every finding in the final hybrid output included a suggested fix. Therefore, repair suggestion coverage was 81 out of 81 findings or 100 percent.

Table 6: Repair suggestion coverage

Metric	Value
Total Findings	81
Findings with suggested fixes	81
Repair suggestion coverage	100%

This is important because the goal of NetConfigGuard is not only to detect insecure configurations but also to provide actionable guidance. For example, SSH findings suggest disabling root login or password authentication, firewall findings suggest restricting access to trusted IP ranges, Docker findings suggest avoiding public dataset port exposure and Nginx findings suggest adding summary metrics and charts.

Discussion:

The results show that NetConfigGuard successfully implements a lightweight and explainable workflow for detecting insecure LLM generated network configurations. The system scanned all 30 configuration files with deterministic rules, incorporated Gemini 2.5 Flash model findings where the free-tier API allowed successful calls, merged duplicates and produced summary metrics and charts. The most important observation is the insecure files produced significantly more findings than the secure aware files. This supports the idea that configuration wording and security intent matter but also that verification remains valuable. Even when a configuration is intended to be secure, a scanner can still provide useful hardening recommendations. The hybrid design also proved useful. The rule checker provided reliable coverage for known high risk patterns. The Gemini model analyzer provided semantic support and identical additional issues beyond the deterministic rule set, such as weak secrets or broader configuration tasks that may not be captured by simple pattern matching. The hybrid detector prevented duplicate counting and identified cases where both components agreed. This design is more reliable than relying only on an LLM, while still preserving an AI-assisted component consistent with the project motivation. The project also has limitations. First, the dataset is small and synthetic although it is appropriate for the research prototype. Second, the rule set is intentionally limited and focuses on common, explainable network security patterns rather than exhaustive security coverage. Third, the Gemini API experiment was constrained by free-tier quota, so not every configuration received a successful Gemini response during the full run. Finally, the system provides repair suggestions but it does not automatically rewrite and validate every configuration file. Despite these limitations, NetConfigGuard provides a meaningful prototype for studying how insecure LLM generated network configurations can be detected and mitigated before deployment.

Conclusion:

This project presented NetconfigGuard, a model-assisted framework for verifying and repairing LLM generated network security configurations. The project was motivated by the risk that LLMs may generate configurations that appear functional but contain insecure settings. NetConfigGuard addresses this problem using a hybrid approach that combines Gemini 2.5 Flash model based analysis, deterministic rule

based verification, finding aggregation, severity classification and repair suggestions. The implementation analyzed 30 configuration files across SSH, firewall, Nginx, Docker Compose and Kubernetes categories. The system produced 81 unique hybrid findings, including 27 high severity and 23 critical findings. The results showed that insecure configurations produced far more findings than secure configurations, supporting the need for verification before deployment. All findings included repair suggestions, making the output actionable rather than only descriptive. Overall, NetConfigGuard demonstrates a practical and explainable approach for detecting and reducing insecure settings in LLM generated framework configurations.

References:

- [1] M. Nazzal, I. Khalil, A. Khreishah, and N. H. Phan, “PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models (LLMs),” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), Salt Lake City, UT, USA, 2024.
- [2] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, “Large Language Model guided Protocol Fuzzing,” in Proc. Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA, 2024.
- [3] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing,” in Proc. 33rd USENIX Security Symposium, Philadelphia, PA, USA, 2024.